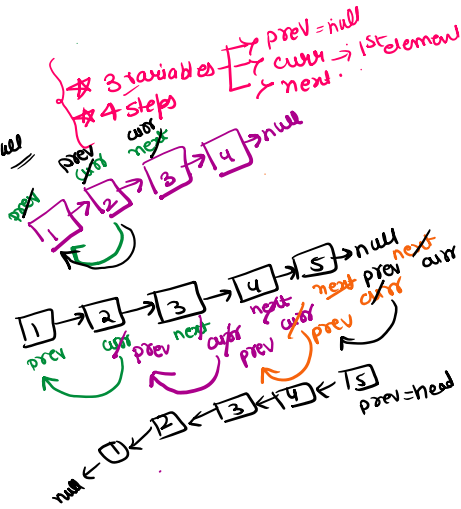


Reverse a LinkedList

```

public void reverse() {
    Node prev = null;
    Node curr = head;
    while (curr != null) {
        Node next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    head = prev;
}

```

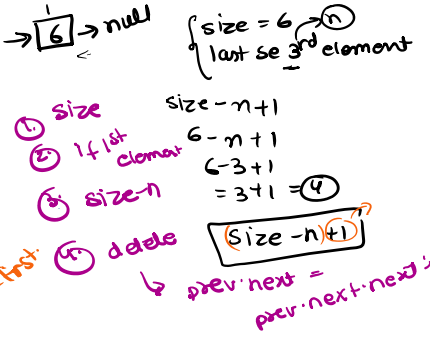


Find and Remove the node from end

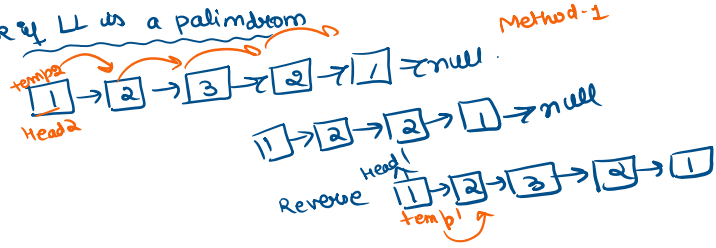
```

public void deleteFromEnd(int n) {
    int sz = 0;
    Node temp = head;
    while (temp != null) {
        temp = temp.next;
        sz++;
    }
    if (n == sz) {
        head = head.next;
        return;
    }
    Node prev = head;
    int i = 1;
    while (i < find) {
        prev = prev.next;
        i++;
    }
    prev.next = prev.next.next;
}

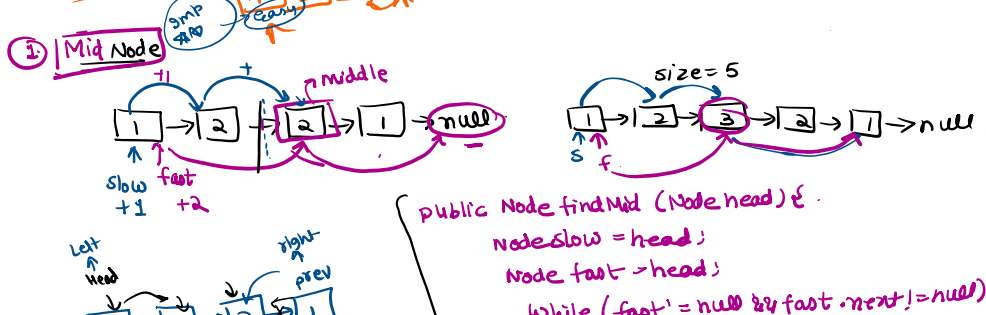
```

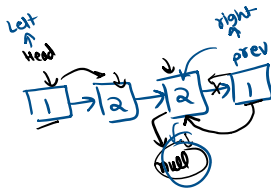


Check if LL is a palindrom



Method-2





```

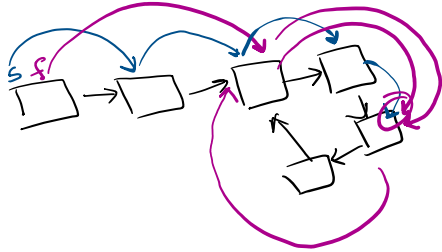
nodeSlow = head;
nodeFast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
return slow;

```

- ① find mid
- ② Reverse the 2nd half
- ③ compare.

ex. point $\rightarrow S = f$ **loop**

Detect a loop/cycle in a LL



$S = f$ **return true**

public boolean isCycle() {

Node slow = head;

Node fast = head;

while (fast != null && fast.next != null) {

slow = slow.next;

fast = fast.next.next;

if (slow == fast) {

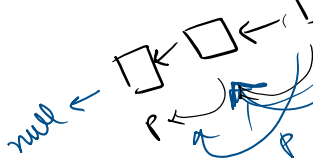
return true;

}

return false;



$F = S$



- ① find the loop
- ② slow $\rightarrow$ head
- slow++
- fast++

- ③ slow = fast
- prev.next = null

